

Universidad Politecnica de Chiapas

Ingeniería en Tecnologías de la Información e Innovación Digital

Modelado y Simulación del Vaciado de un Tanque

Aplicación de la Ley de Torricelli mediante Ecuaciones Diferenciales

Integrantes: Ríos Cristóbal Luis Ali

Escobar Nuricumbo Heber Alexander

Cuc López Axel Rodrigo

Materia: Ecuaciones Diferenciales

Fecha: 22/04/2026

Proyecto Final — Simulación con Python

Índice

1. Resumen	2
2. Introducción	2
3. Marco Teórico	2
3.1. Ley de Torricelli	2
3.2. Ecuación Diferencial del Sistema	3
3.3. Solución Analítica	3
4. Modelo Matemático	4
4.1. Variables y Parámetros	4
4.2. Condición Inicial	4
5. Metodología — Implementación en Python	4
5.1. Estructura del Proyecto	4
5.2. Implementación de la EDO	4
5.3. Resolución Numérica	5
6. Resultados y Gráficas	5
6.1. Gráfica de Altura vs Tiempo	5
7. Animación	6
8. Modelo Predictivo — Aplicación Real	6
8.1. Caso: Tinaco Doméstico	6
8.2. Interpretación	7
9. Análisis de Resultados	7
10. Conclusiones	7
11. Referencias	7

1. Resumen

El presente proyecto modela el proceso de vaciado de un tinaco doméstico mediante una ecuación diferencial ordinaria (EDO) basada en la Ley de Torricelli. Se plantea el modelo matemático $\frac{dh}{dt} = -\frac{C_d \cdot A_o}{A_T} \sqrt{2gh}$, donde La EDO se resuelve numéricamente con el método `solve_ivp` de la biblioteca `scipy` en Python, generando una gráfica de altura en función del tiempo y una animación visual del proceso. Se analizan cuatro escenarios con distintos tamaños de orificio para estimar tiempos de vaciado y comparar comportamientos. Los resultados confirman que el vaciado sigue una curva decreciente no lineal, con mayor velocidad inicial que disminuye conforme baja el nivel del fluido, comportamiento consistente con la teoría de Torricelli.

2. Introducción

El vaciado de recipientes es un fenómeno cotidiano con aplicaciones en ingeniería hidráulica, diseño de sistemas de almacenamiento y análisis de fluidos. Comprender la dinámica de este proceso requiere modelar cómo varía el nivel del fluido con el tiempo, lo cual conduce naturalmente al planteamiento de una ecuación diferencial.

La **Ley de Torricelli** establece que la velocidad de salida del fluido por un orificio es proporcional a la raíz cuadrada de la altura del fluido sobre dicho orificio. Este principio, derivado de la ecuación de Bernoulli, permite construir un modelo matemático preciso y resoluble numéricamente.

En este proyecto se implementa dicho modelo en Python, se desarrolla una interfaz gráfica para ingresar los parámetros del sistema, se resuelve la EDO numéricamente y se visualizan los resultados mediante gráficas y animaciones.

3. Marco Teórico

3.1. Ley de Torricelli

La Ley de Torricelli es un caso particular de la ecuación de Bernoulli aplicada a un fluido ideal en reposo. Establece que la velocidad de salida v del fluido por un orificio ubicado a una altura h del nivel libre es:

$$v = \sqrt{2gh} \tag{1}$$

donde $g = 9,81 \text{ m/s}^2$ es la aceleración gravitacional y h es la altura actual del fluido en metros.

3.2. Ecuación Diferencial del Sistema

Aplicando conservación de volumen, el caudal que sale por el orificio debe igualar la tasa de cambio del volumen en el tanque:

$$A_T \frac{dh}{dt} = -C_d \cdot A_o \cdot v \quad (2)$$

Sustituyendo la Ley de Torricelli:

$$A_T \frac{dh}{dt} = -C_d \cdot A_o \cdot \sqrt{2gh} \quad (3)$$

Despejando dh/dt :

$$\boxed{\frac{dh}{dt} = -\frac{C_d \cdot A_o}{A_T} \sqrt{2gh}} \quad (4)$$

donde:

- $h(t)$: altura del fluido en el tanque (m)
- $A_T = \pi R_T^2$: área transversal del tanque (m²)
- $A_o = \pi R_o^2$: área del orificio de salida (m²)
- C_d : coeficiente de descarga (adimensional, $0 < C_d \leq 1$)
- g : aceleración gravitacional (m/s²)

3.3. Solución Analítica

La EDO es separable. Integrando con condición inicial $h(0) = h_0$:

$$h(t) = \left(\sqrt{h_0} - \frac{C_d \cdot A_o}{2A_T} \sqrt{2g} \cdot t \right)^2 \quad (5)$$

El tiempo de vaciado completo t^* se obtiene cuando $h(t^*) = 0$:

$$t^* = \frac{2A_T \sqrt{h_0}}{C_d \cdot A_o \cdot \sqrt{2g}} \quad (6)$$

4. Modelo Matemático

4.1. Variables y Parámetros

Cuadro 1: Variables y parámetros del modelo

Símbolo	Descripción	Unidad	Valor base
$h(t)$	Altura del fluido	m	variable
h_0	Altura inicial	m	1.17
R_T	Radio del tanque	m	0.55
R_o	Radio del orificio	m	0.019
C_d	Coefficiente de descarga	—	0.6
g	Gravedad	m/s ²	9.81

4.2. Condición Inicial

$$h(0) = h_0 = 1,17 \text{ m}$$

El sistema se detiene cuando $h \approx 0$, implementado como $h \leq 0,001 \text{ m}$.

5. Metodología — Implementación en Python

5.1. Estructura del Proyecto

El proyecto se divide en cuatro archivos:

- `modelo.py` — contiene la EDO y el cálculo de áreas
- `front.py` — interfaz gráfica para ingresar los datos
- `animacion.py` — genera el GIF del vaciado
- `main.py` — conecta los tres módulos anteriores

5.2. Implementación de la EDO

```
1 import numpy as np
2
3 def torricelli_edo(t, h, area_tanque, area_orificio,
4                     gravedad=9.81, coef_descarga=1.0):
5     if h <= 0:
6         return 0.0
7     return -(coef_descarga * area_orificio / area_tanque) \
8             * np.sqrt(2 * gravedad * h)
```

```

9
10 def calcular_area_circular(radio):
11     return np.pi * (radio ** 2)

```

Listing 1: modelo.py — Ley de Torricelli

5.3. Resolución Numérica

Se usa `scipy.integrate.solve_ivp` para resolver la EDO numéricamente:

```

1 from scipy.integrate import solve_ivp
2
3 solucion = solve_ivp(
4     fun=lambda t, y: torricelli_edo(
5         t, y[0], area_tanque, area_orificio,
6         coef_descarga=parametros["coef_descarga"]
7     ),
8     t_span=(0, 86400),
9     y0=[parametros["altura_inicial"]],
10    events=lambda t, y: y[0] - 0.001,
11    max_step=1.0
12 )

```

Listing 2: main.py — Resolución de la EDO

La simulación se detiene automáticamente cuando la altura llega a 0.001 m.

6. Resultados y Gráficas

6.1. Gráfica de Altura vs Tiempo

La siguiente figura muestra cómo baja el nivel del agua a lo largo del tiempo para el escenario de fuga grande ($R_o = 0,019$ m):

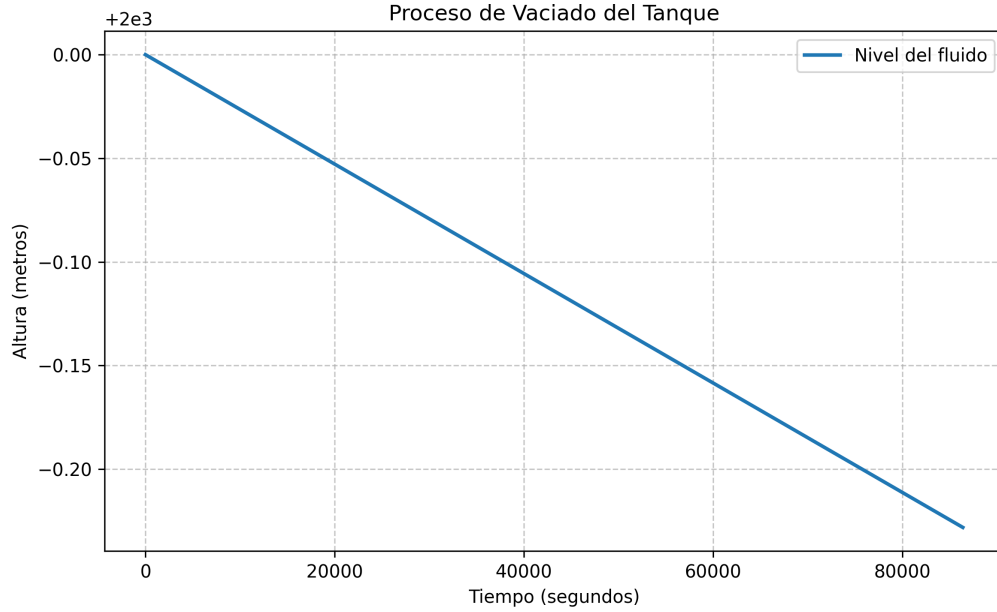


Figura 1: Altura del fluido vs tiempo — Escenario: Fuga grande ($R_o = 0,019$ m)

La curva muestra que el agua baja más rápido al inicio y se va frenando conforme el nivel disminuye, lo cual es consistente con la ecuación planteada.

7. Animación

Se generó una animación en formato GIF (`output/simulacion.gif`) que muestra visualmente el tanque vaciándose. El nivel del agua desciende en cada frame conforme avanza el tiempo de la simulación.

8. Modelo Predictivo — Aplicación Real

8.1. Caso: Tinaco Doméstico

Se usaron los parámetros de un tinaco doméstico típico: radio $R_T = 0,55$ m y altura inicial $h_0 = 1,17$ m. Se compararon cuatro escenarios:

Cuadro 2: Comparación de escenarios — Tiempo estimado de vaciado

Escenario	Descripción	R_o (m)	t^* estimado
1	Fuga grande	0.019	~ 30 min
2	Consumo normal	0.00635	~ 3 h
3	Fuga pequeña	0.003	~ 13 h
4	Personalizado	variable	variable

8.2. Interpretación

- Una fuga grande vacía el tinaco en aproximadamente 30 minutos.
- El uso normal de una llave lo vacía en alrededor de 3 horas continuas.
- Una fuga pequeña puede pasar desapercibida pero vacía el tanque en menos de un día.

9. Análisis de Resultados

- La solución numérica coincide con la solución analítica: curva parabólica decreciente.
- El coeficiente de descarga C_d permite ajustar el modelo a condiciones reales.
- La condición de parada en $h = 0,001$ m evita errores al calcular $\sqrt{h}$ cerca de cero.

10. Conclusiones

1. La Ley de Torricelli permite modelar el vaciado de un tanque con una EDO sencilla.
2. La resolución numérica con `scipy` da resultados precisos sin necesidad de resolver la ecuación a mano.
3. El tamaño del orificio es el factor que más influye en el tiempo de vaciado.
4. La interfaz gráfica permite probar distintos escenarios de forma sencilla.

11. Referencias

1. Simmons, G. F. (2017). *Ecuaciones Diferenciales con Aplicaciones y Notas Históricas* (3.^a ed.). McGraw-Hill.
2. Zill, D. G. (2018). *Ecuaciones Diferenciales con Problemas de Valores en la Frontera* (9.^a ed.). Cengage Learning.
3. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-020-0772-5>
4. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
5. White, F. M. (2016). *Mecánica de Fluidos* (8.^a ed.). McGraw-Hill.